1 The Story of U-Net: A Journey through Convolutions and Upsampling

Once upon a time, deep learning researchers set out to solve a great challenge in the world of image segmentation: how to capture both fine details and large-scale context in images, particularly medical images. Enter **U-Net**, a special kind of neural network designed to handle this problem with precision and efficiency. But how does it work? Let's journey into its inner workings to understand how U-Net applies filters, extracts features, and reconstructs high-resolution images through **upsampling**.

1.1 The Input: Where the Journey Begins

Imagine you have an image, or some other data, with **two channels**, each of size 4×4. These channels might represent different kinds of information. For example, in a medical image, one channel could highlight blood vessels, while the other shows tissue density.

The **first channel** of our input image, $\mathbf{I_1}$, might look like this:

$$\mathbf{I_1} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Meanwhile, the **second channel**, $\mathbf{I_2}$, could look like this:

$$\mathbf{I_2} = \begin{bmatrix} 4 & 3 & 2 & 1 \\ 8 & 7 & 6 & 5 \\ 12 & 11 & 10 & 9 \\ 16 & 15 & 14 & 13 \end{bmatrix}$$

These two channels represent our input data, and now the adventure begins—let's see what U-Net does with this information.

1.2 Filters: The Magical Tools

To make sense of these input channels, we apply filters to extract useful features. Imagine we have two filters, each designed to create a unique perspective on the input data:

Filter 1 has one part that looks at the first channel, $\mathbf{I_1}$, and another part that looks at the second channel, $\mathbf{I_2}$.

$$\mathbf{W_1^1} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{14} & w_{15} & w_{16} \\ w_{17} & w_{18} & w_{19} \end{bmatrix} \quad \text{and} \quad \mathbf{W_1^2} = \begin{bmatrix} w_{21} & w_{22} & w_{23} \\ w_{24} & w_{25} & w_{26} \\ w_{27} & w_{28} & w_{29} \end{bmatrix}$$

Filter 2 is similar, but it has different weights, offering a new way to interpret the data from both channels.

$$\mathbf{W_2^1} = \begin{bmatrix} w_{31} & w_{32} & w_{33} \\ w_{34} & w_{35} & w_{36} \\ w_{37} & w_{38} & w_{39} \end{bmatrix} \quad \text{and} \quad \mathbf{W_2^2} = \begin{bmatrix} w_{41} & w_{42} & w_{43} \\ w_{44} & w_{45} & w_{46} \\ w_{47} & w_{48} & w_{49} \end{bmatrix}$$

These filters are our magical tools, and they will soon be applied to the input data to extract meaningful features.

1.3 The Convolution Process: Summing the Magic

Once we have our filters, we apply them to the input data through a process called **convolution**. Convolution lets us extract information from the input channels by sliding the filters over each channel and multiplying the corresponding values.

Let's start with **Output Channel 1**. To compute the top-left value of this output, we apply **Filter 1** to both input channels:

- First, we take a 3×3 patch from **Input Channel 1**:

$$\begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}$$

  Then, we multiply each value by the corresponding weight in the filter, sum them up, and we get the **first contribution** to the output.

- Next, we take a similar 3×3 patch from **Input Channel 2**:

$$\begin{bmatrix} 4 & 3 & 2 \\ 8 & 7 & 6 \\ 12 & 11 & 10 \end{bmatrix}$$

  We apply the second part of **Filter 1** to this patch and get the **second contribution**.

- The two contributions are summed, and we have the top-left value of **Output Channel 1**:

  $\text{Output}_{(1,1)}^{(1)} = (\text{Contribution from Channel 1}) + (\text{Contribution from Channel 2})$

We repeat this process across all spatial positions to fill the entire **Output Channel 1**. The same steps are followed for **Output Channel 2**, using **Filter 2**. This is the essence of how convolution works on multi-channel data.

1.4 ReLU Activation: Adding Some Non-Linearity

After computing the output feature maps, U-Net adds a bit of magic with the **ReLU activation function**. This function is simple: it turns any negative values into zeros. This helps the network focus on the most important features and ignore unnecessary noise. After applying ReLU, our output feature maps might look something like this:

Output Channel 1:

$$\text{Output}_1 = \begin{bmatrix} \text{ReLU}\left(\text{Output}^{(1)}_{(1,1)}\right) & \text{ReLU}\left(\text{Output}^{(1)}_{(1,2)}\right) \\ \text{ReLU}\left(\text{Output}^{(1)}_{(2,1)}\right) & \text{ReLU}\left(\text{Output}^{(1)}_{(2,2)}\right) \end{bmatrix}$$

Output Channel 2:

$$\text{Output}_2 = \begin{bmatrix} \text{ReLU}\left(\text{Output}^{(2)}_{(1,1)}\right) & \text{ReLU}\left(\text{Output}^{(2)}_{(1,2)}\right) \\ \text{ReLU}\left(\text{Output}^{(2)}_{(2,1)}\right) & \text{ReLU}\left(\text{Output}^{(2)}_{(2,2)}\right) \end{bmatrix}$$

With these activated output channels, the network is now ready to move to the next stage, where the input size will change.

1.5 Generalizing to Any Number of Channels

In real-world applications, you won't be limited to just two input channels or two output channels. The beauty of U-Net lies in its ability to scale and handle images with more than two channels (e.g., RGB images with 3 channels) and any number of output channels for different tasks.

To go from an arbitrary number of input channels (C_{in}) to an arbitrary number of output channels (C_{out}):

1. **Apply filters** across all input channels. For each output channel, use a separate filter.
2. **Sum the contributions** from all input channels after applying the filter.
3. **Adjust stride and padding** to control the size of the output feature maps.

The formula to calculate the output size is:

$$H_{out} = \left\lfloor \frac{H_{in} - f + 2p}{s} \right\rfloor + 1$$

where f is the filter size, p is the padding, and s is the stride.

1.6 Upsampling: Bringing Back the Resolution

In the second half of U-Net, we need to **upsample** the feature maps back to their original size, since convolutions and pooling reduce the spatial dimensions. **Upsampling** works like the reverse of max-pooling, where we take a lower-resolution feature map and expand it by inserting zeros between the values.

For example, suppose we start with a small feature map:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

If we upsample this map by inserting zeros, we get:

$$\mathbf{X}_{\mathbf{upsample}} = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

After inserting zeros, we apply a **convolution** to fill in the blanks and smooth the feature map. This upsampling process restores the resolution of the feature maps, allowing U-Net to generate detailed, high-resolution outputs.

1.7 Comparison to Max-Pooling

Max-pooling reduces the spatial dimensions by selecting the maximum value from a region. For example, applying max-pooling to a 4×4 matrix with a 2×2 filter might look like this:

$$\text{Input Matrix} = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 11 & 10 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

After max-pooling, the result is:

$$\text{Max-Pooled Matrix} = \begin{bmatrix} 6 & 8 \\ 14 & 16 \end{bmatrix}$$

When upsampling, you can take this max-pooled output and reverse the downsampling process by inserting zeros:

$$\text{Upsampled Matrix (Zero-Insertion)} = \begin{bmatrix} 6 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 \\ 14 & 0 & 16 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Upsampling thus helps restore the original resolution of the image.

1.8 Transposed Convolution (Deconvolution)

Another common upsampling method is **transposed convolution**, which directly learns how to upsample by applying a reverse convolution operation, without the need to insert zeros manually. Transposed convolution is used in many deep learning models to ensure smooth transitions from low-resolution to high-resolution feature maps.

1.9 Loss Function: Measuring the Success of U-Net

For tasks like image segmentation, where U-Net is commonly applied, a crucial part of the model is the **loss function**, which guides the training process. Suppose we have a problem where we want to classify each pixel into **two classes** (e.g., foreground and background). In this case, the loss function typically used is the **binary cross-entropy loss**.

Binary cross-entropy compares the predicted probability for each pixel (whether it belongs to class 1 or class 0) with the true label. For a single pixel, the binary cross-entropy loss is defined as:

$$\text{Loss}(y, \hat{y}) = -\left(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right)$$

where:

- y is the true label of the pixel (either 0 or 1),
- $\hat{y}$ is the predicted probability for the pixel being in class 1.

When applied across all the pixels in the image, the loss function sums or averages the individual losses to provide an overall measure of how well the model is performing. The goal during training is to **minimize this loss**, improving the accuracy of pixel classification over time.

In U-Net, the combination of **convolution**, **upsampling**, and an effective **loss function** enables the network to learn to produce highly accurate pixel-wise classifications, essential for tasks like medical image segmentation.

1.10 Conclusion: The Power of U-Net

Through carefully designed filters, convolution, and upsampling, U-Net brings together the best of both worlds: **global context** and **local details**. By using convolutions to extract features and upsampling to restore resolution, U-Net has become a powerful tool for tasks like image segmentation, where precise boundaries and large-scale understanding are crucial.

By minimizing the appropriate **loss function**, U-Net learns to differentiate between classes and create detailed segmentations, making it indispensable for many deep learning applications, especially in medical image analysis.

2 Why the U-Net Structure Was Used

The structure of U-Net is designed to capture both fine details (local features) and large-scale contextual information, which is critical for tasks like image segmentation, especially in medical images. Each part of the U-Net architecture plays a distinct role in achieving this balance between high-level context and fine-level segmentation accuracy. Below, we discuss why each part of U-Net is important, and then suggest possible improvements.

2.1 Contracting Path (Encoder) - Feature Extraction

The left side of U-Net, or the **contracting path**, functions similarly to traditional convolutional networks:

- **Multiple Convolutional Layers**: These layers extract local features at various scales. Each convolution operation, followed by ReLU activation, allows the model to capture patterns like edges, textures, and more complex structures as we go deeper.
- **Max-Pooling**: Max-pooling layers after each pair of convolutions progressively reduce the spatial resolution while increasing the depth of feature maps. This allows the network to focus on higher-level, abstract features by summarizing regions of the image.

Why it's important: The contracting path extracts deep feature representations while gradually reducing the image size. This allows U-Net to build a hierarchy of features, starting from low-level textures to more abstract shapes and objects.

2.2 Bottleneck (Bottom Layer) - High-Level Features

At the bottom of U-Net, the spatial dimensions are the smallest, but the number of channels (or feature maps) is the largest. The purpose of this layer is to act as a **bridge** between the encoder and decoder.

Why it's important: The bottleneck ensures that the network can learn high-level, abstract features that summarize the entire input image. By using many feature maps, the network can capture various important global contexts.

2.3 Expansive Path (Decoder) - Upsampling and Localization

The right side of U-Net, or the **expansive path**, is responsible for recovering the spatial resolution lost during the contracting phase:

- **Upsampling**: The expansive path uses **upsampling** (via transposed convolutions or other techniques) to gradually increase the spatial resolution back to the original image size.

- **Concatenation of Feature Maps**: Skip connections concatenate feature maps from the contracting path to the corresponding layers in the expansive path. This allows the network to reuse the fine-grained details from earlier layers, while also leveraging the high-level features learned deeper in the network.

Why it's important: The expansive path reconstructs the high-resolution output using both the upsampled feature maps and the detailed information from the earlier layers. This process allows the network to localize and identify boundaries in the image with high accuracy.

2.4 Final 1x1 Convolution - Pixel-Wise Classification

The final layer of U-Net is a **1x1 convolution**, which reduces the number of output channels to the number of classes (e.g., foreground and background).

Why it's important: This step performs the **pixel-wise classification**, where each pixel is assigned a class based on the learned features. The 1x1 convolution ensures that the network can output the correct number of classes for each pixel, making U-Net suitable for segmentation tasks.

2.5 Large Number of Feature Channels in the Upsampling Path

One of the important modifications in U-Net is the use of a **large number of feature channels** in the upsampling path. Even as the spatial resolution increases in the upsampling process, the number of channels remains high. This allows the network to propagate **contextual information** from the contracting path to higher-resolution layers in the expansive path.

Why it's important: This ensures that when the network upsamples the image, it still has access to high-level context, which helps in accurately segmenting fine details and boundaries. The large number of channels allows for richer representations even in the higher-resolution stages.

2.6 Symmetric U-Net Architecture

The expansive path is more or less symmetric to the contracting path, forming the characteristic **U-shaped architecture**. This symmetry is essential because it allows the network to **combine low-resolution contextual information** with **high-resolution spatial details** through **skip connections**.

Why it's important: This symmetry ensures that the information lost during downsampling is recovered through the skip connections and upsampling, leading to more accurate and detailed segmentations.

2.7 Valid Convolution and Overlap-Tile Strategy

In U-Net, the network uses only the **valid part of each convolution**, which means that the output segmentation map only contains the pixels where the full

context from the input image is available. This is done by not using padding in the convolutional layers.

To segment larger images, U-Net uses an **overlap-tile strategy**, where the input image is divided into smaller tiles. The network processes each tile separately, and the predictions for each tile are then combined to form the full segmentation map.

Why it's important: This allows U-Net to process arbitrarily large images, which is crucial for applications like medical imaging. Without this tiling strategy, the resolution would be limited by the available GPU memory. The **mirroring strategy** is used to handle predictions near the borders of the image, ensuring that the network has enough context even at the edges.

2.8 GPU Memory Efficiency

The overlap-tile strategy is also important for **managing GPU memory**. Large images require significant memory to process, and attempting to segment an entire large image at once could exceed the available GPU memory. By processing smaller tiles of the image, U-Net can handle larger inputs more efficiently.

3 Tiling and Mirroring Strategy in U-Net

3.1 Handling Large Images with Tiling

In practice, U-Net is often used on **large images**, which may not fit into GPU memory in a single pass. To handle this, U-Net divides the image into **tiles** (sub-images), processes them independently, and then stitches the results back together. This technique is known as the **overlap-tile strategy**.

For instance, consider a large image of size 1000×1000 pixels, which is too large to process directly. Instead, the image is divided into smaller overlapping tiles of size **572x572** (including mirrored borders).

3.2 Exact Sizes of the Tiles in a 1000x1000 Image

In a **1000x1000** image, divided into **3x3 tiles**, each tile starts as **572x572** (after mirroring). After applying valid convolutions in U-Net, the effective size of each tile is reduced to **388x388**. The **92-pixel overlap** ensures smoother stitching of the tiles. Below are the exact positions and sizes of the tiles:

- **Tile 1 (Top-Left)**:
  - Input: $(0, 572) \times (0, 572)$
  - Output: $(0, 388) \times (0, 388)$
- **Tile 2 (Top-Middle)**:
  - Input: $(0, 572) \times (296, 868)$

  - Output: $(0, 388) \times (296, 684)$

- **Tile 3 (Top-Right)**:
  - Input: $(0, 572) \times (608, 1000)$
  - Output: $(0, 388) \times (608, 996)$

- **Tile 4 (Middle-Left)**:
  - Input: $(296, 868) \times (0, 572)$
  - Output: $(296, 684) \times (0, 388)$

- **Tile 5 (Middle-Middle)**:
  - Input: $(296, 868) \times (296, 868)$
  - Output: $(296, 684) \times (296, 684)$

- **Tile 6 (Middle-Right)**:
  - Input: $(296, 868) \times (608, 1000)$
  - Output: $(296, 684) \times (608, 996)$

- **Tile 7 (Bottom-Left)**:
  - Input: $(608, 1000) \times (0, 572)$
  - Output: $(608, 996) \times (0, 388)$

- **Tile 8 (Bottom-Middle)**:
  - Input: $(608, 1000) \times (296, 868)$
  - Output: $(608, 996) \times (296, 684)$

- **Tile 9 (Bottom-Right)**:
  - Input: $(608, 1000) \times (608, 1000)$
  - Output: $(608, 996) \times (608, 996)$

3.3 Stitching the Tiles Back Together

After processing all the tiles independently, the outputs are stitched together to form the full segmentation map. The **overlap-tile strategy** ensures smooth transitions between tiles, especially near the edges, where the mirroring ensures that every pixel has sufficient context. The following techniques can be used to handle conflicts when different tiles provide conflicting predictions for the same pixel:

- **Averaging the Predictions (Soft Voting)**: Average the probability predictions across all tiles before applying the threshold for 0 or 1.

- **Majority Voting (Hard Voting)**: Take the majority vote from the predictions of all tiles that overlap on the same pixel.
- **Weighted Voting**: Assign higher weights to predictions from the center of the tile and lower weights to those near the edges to avoid boundary artifacts.

3.4 Why Tiling and Mirroring Are Important

- **Efficient Memory Usage**: Dividing large images into tiles allows U-Net to process them efficiently, without exceeding GPU memory limits. - **Accurate Border Predictions**: Mirroring ensures that predictions near the borders of each tile are as accurate as those in the center by providing sufficient context around the edges. - **Seamless Segmentation**: Overlap between tiles helps ensure that the stitched-together segmentation map is smooth and continuous, with no visible seams or artifacts.

Mirroring for Border Context

To handle predictions near the borders of each tile, U-Net uses a **mirroring strategy**. This means the image is reflected at its borders, so the network has sufficient context even for pixels near the edges. For example, if the original image is **388x388**, mirroring would extend it to **572x572**, providing the necessary context for accurate segmentation.

Stitching the Tiles Back Together

After processing all the tiles independently, the outputs are stitched together to form the full segmentation map. If multiple tiles predict different values for the same pixel (due to overlap), U-Net can handle this through several strategies:

- **Averaging the Predictions (Soft Voting)**: Averaging the probability predictions across all tiles before applying the threshold for 0 or 1.
- **Majority Voting (Hard Voting)**: Taking the majority vote from the predictions of all tiles that overlap on the same pixel.
- **Weighted Voting**: Assigning higher weights to pixels near the center of the tile and lower weights to those near the edges to avoid boundary artifacts.

3.5 Why Tiling and Mirroring Are Important

- **Efficient Memory Usage**: Dividing large images into tiles allows U-Net to process them efficiently, without exceeding GPU memory limits. - **Accurate Border Predictions**: Mirroring ensures that predictions near the borders of each tile are as accurate as those in the center by providing sufficient context around the edges. - **Seamless Segmentation**: Overlap between tiles helps ensure that the stitched-together segmentation map is smooth and continuous, with no visible seams or artifacts.

3.6 Stitching Example with Conflict Resolution

Consider four overlapping tiles that make conflicting predictions for a particular pixel (e.g., two tiles predict 1, and two predict 0). A common solution is to average the probabilities across the tiles and apply a threshold (e.g., 0.5) to resolve the conflict.

For example: - Tile 1: Probability = 0.7 - Tile 2: Probability = 0.3 - Tile 3: Probability = 0.9 - Tile 4: Probability = 0.4

The average probability is:

$$\frac{0.7 + 0.3 + 0.9 + 0.4}{4} = 0.575$$

Applying a threshold of 0.5 results in a final prediction of **1** for that pixel.

4 Conclusion: The Power of U-Net

Through carefully designed filters, convolution, and upsampling, U-Net brings together the best of both worlds: **global context** and **local details**. By using convolutions to extract features and upsampling to restore resolution, U-Net has become a powerful tool for tasks like image segmentation, where precise boundaries and large-scale understanding are crucial.

By minimizing the appropriate **loss function**, U-Net learns to differentiate between classes and create detailed segmentations, making it indispensable for many deep learning applications, especially in medical image analysis.